

الفصل الثالث

معمارية نظام إدارة المشاريع الجامعية (PMS)

Project Management System — System Architecture

React 19 · Laravel 10 · MySQL · Sanctum · Google Gemini
Multi-Tenant · Service Layer · REST API

فهرس المحتويات

1. مقدمة وأهداف المعمارية
2. نظرة عامة على النظام (System Context)
3. النمط المعماري العام
4. معمارية Backend — Laravel
5. معمارية Frontend — React
6. معمارية Multi-Tenancy
7. معمارية المصادقة والأمان
8. معمارية الذكاء الاصطناعي
9. معمارية البيانات
10. أنماط التصميم ومبادئ SOLID
11. النشر والبيئة التشغيلية
12. المتطلبات غير الوظيفية
13. مقارنة البدائل ومبررات الاختيار
14. القيود والتحسينات المستقبلية
15. الخلاصة

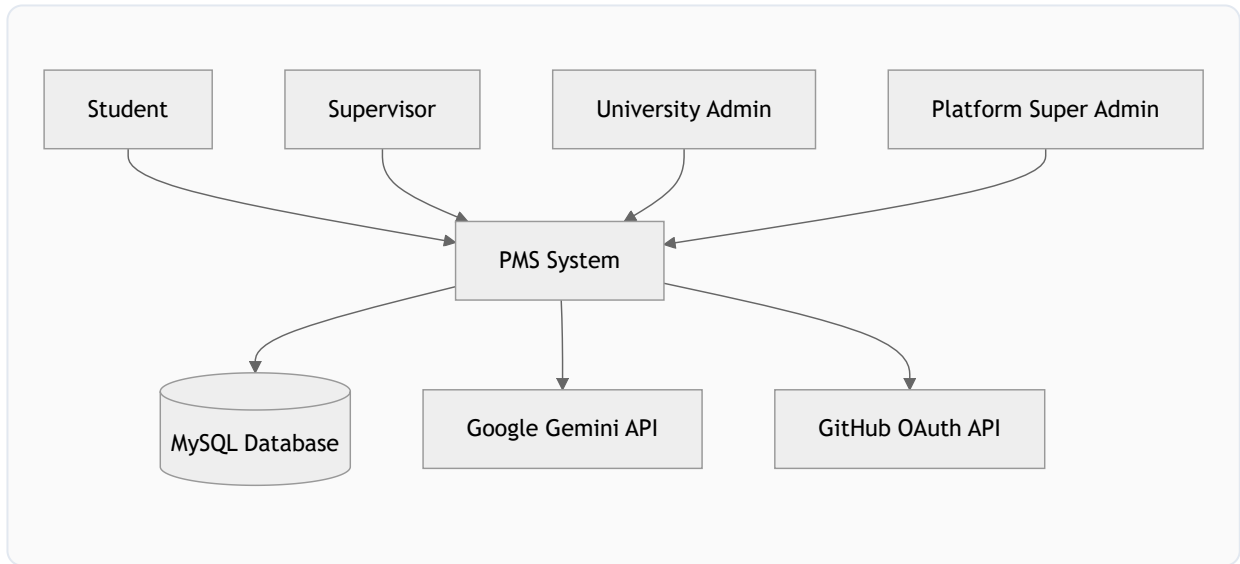
1. مقدمة وأهداف المعمارية

يهدف نظام (PMS) (Project Management System) إلى إدارة مشاريع التخرج الجامعية ضمن بيئة متعددة الجامعات (Multi-Tenant)، حيث تستضيف المنصة عدة جامعات كـ tenants مستقلين منطقياً، مع إمكانية إدارة مركزية على مستوى المنصة. تسعى المعمارية إلى تحقيق الأهداف التالية:

- فصل الاهتمامات (Separation of Concerns): فصل العرض عن منطق الأعمال عن البيانات.
- قابلية الصيانة: تنظيم الكود في طبقات وخدمات (Services) يسهل تطويرها واختبارها.
- عزل البيانات (Data Isolation): ضمان أن كل جامعة ترى بياناتها فقط.
- مرونة الأدوار (RBAC): دعم student، supervisor، admin، super_admin بصلاحيات مختلفة.
- قابلية التوسع التدريجي: بنية Monolith قابلة للفصل لاحقاً دون إعادة بناء كاملة.
- دمج AI: ربط Google Gemini كخدمة خارجية دون coupling مباشر في Controllers.

2. نظرة عامة على النظام (System Context)

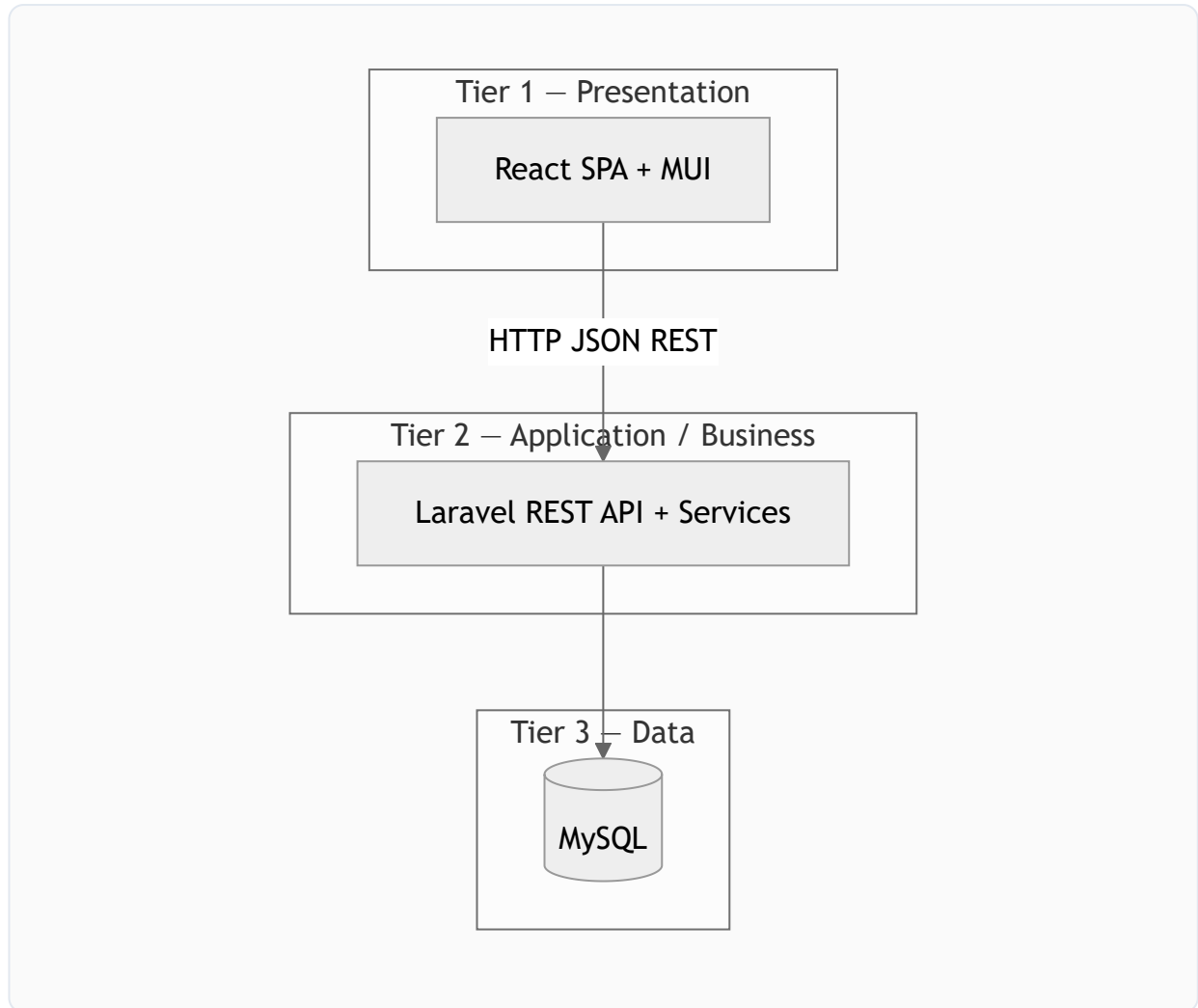
يتفاعل النظام مع الفئات والأنظمة الخارجية التالية:



الدور	فاعل / نظام خارجي
إنشاء مشاريع، مهام، AI ideation، دعوات	Student
الإشراف على مشاريع، قبول/رفض دعوات	Supervisor
إدارة مستخدمي الجامعة، الموافقة على التسجيل	University Admin
إدارة الجامعات والمنصة بالكامل	Platform Super Admin
تخزين دائم لكل بيانات الأعمال	MySQL
اقتراح أفكار مشاريع وتوليد مهام	Google Gemini
OAuth وربط مستودعات المشاريع	GitHub

3. النمط المعماري العام

يعتمد PMS على نمط **Client-Server** ثلاثي الطبقات (**Three-Tier**) مع فصل Frontend عن Backend:



3.1 تصنيف المعمارية

البعد	النمط المستخدم	ملاحظة
نمط التطبيق	Monolithic Backend + SPA Frontend	ليس Microservices
Backend	Layered Architecture + MVC	Laravel convention
Business Logic	Service Layer Pattern	Services بين Model و Controller
Frontend	Component-Based SPA	React + React Router
Communication	REST over HTTP	JSON payloads
Auth	Stateless Token-Based	Laravel Sanctum
Multi-Tenancy	Shared DB, Row-Level Isolation	university_id + TenantScope
AI	External Service Integration	HTTP to Gemini via Services

التسمية الأكاديمية الموصى بها:

Three-Tier Client–Server Architecture with Layered Backend (MVC + Service Layer),
Multi-Tenant Row-Level Isolation, Stateless Token Authentication, and Component-
Based SPA Frontend

4. معمارية Laravel Backend

4.1 هيكل المجلدات

```

backend last/
├─ app/Http/Controllers/      ← HTTP handlers (thin)
├─ app/Http/Middleware/      ← Auth, Role, Tenant, Status
├─ app/Services/              ← Business logic
├─ app/Repositories/          ← ProjectRepository (partial)
├─ app/Models/                ← Eloquent + TenantScope
├─ app/Support/ProjectAccess.php
├─ config/ai.php              ← AI prompt templates
└─ routes/api.php             ← API entry points
  
```

4.2 تدفق الطلب (Request Pipeline)



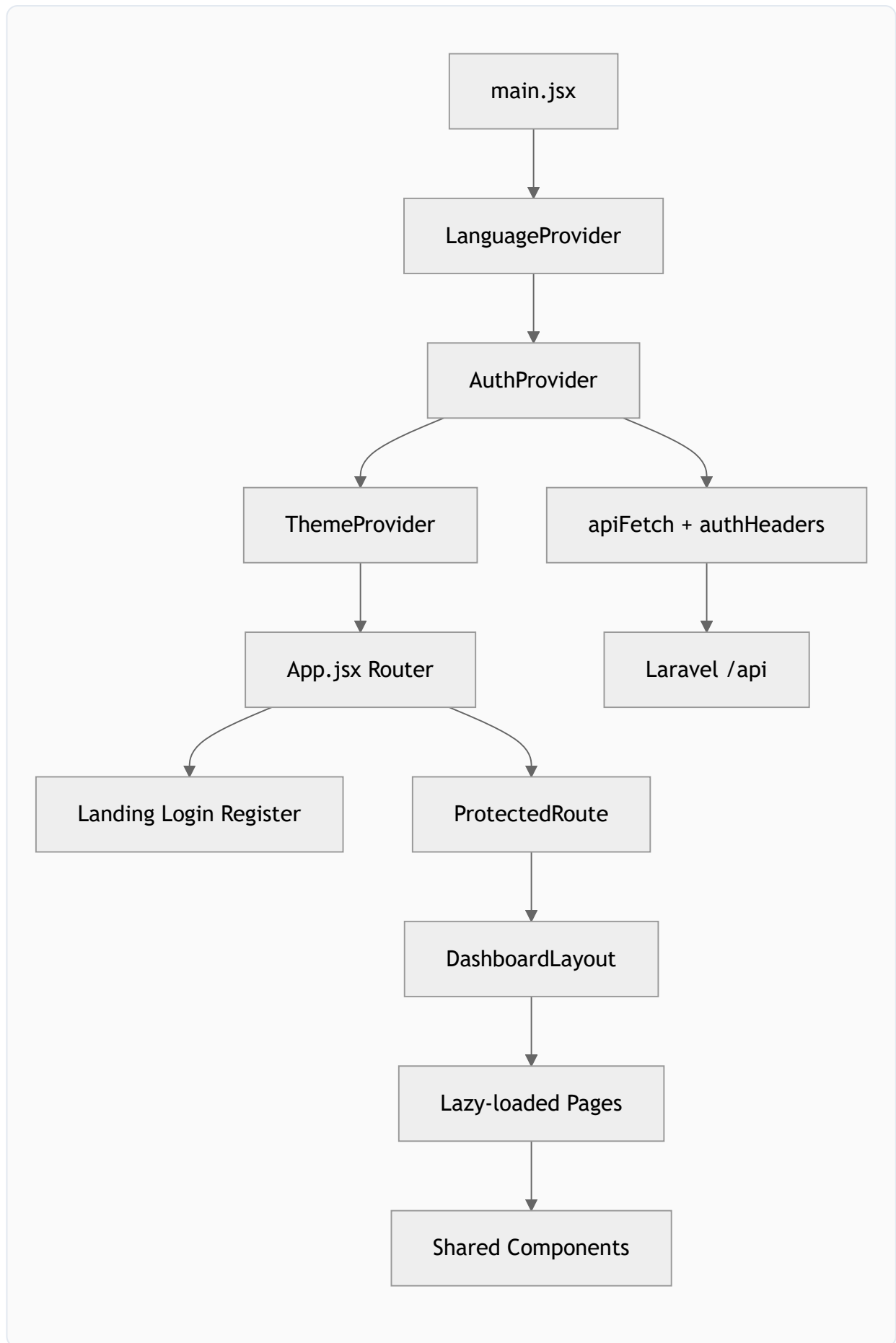
4.3 Service Layer — الخدمات الرئيسية

المسؤولية	Service
تسجيل، دخول، tokens، تحديث الملف	AuthService
CRUD مشاريع، تقدم، أنشطة	ProjectService
CRUD مهام، إشعارات	TaskService
تعليقات المشروع والمهمة	CommentService
دعوات مشرف وطالب	InvitationService
إشعارات قاعدة البيانات	NotificationService
اقتراح أفكار عبر Gemini	AIdeationService
توليد مهام عبر Gemini	AITaskService
طلبات reset من admin الجامعة	PasswordResetHelpService
حذف مستخدم وتنظيف علاقاته	UserDeletionService

يتبع كل Controller مبدأ **Thin Controller**: يتحقق من المدخلات ويستدعي Service، بينما Service يحمل القواعد والعمليات المعقدة.

5. معمارية React — Frontend

5.1 بنية التطبيق



Route Guards 5.2 — حماية المسارات

Guard	الوظيفة
ProtectedRoute	يتطلب token + status=active
SuperAdminRoute	super_admin فقط — إدارة المنصة
TenantRoute	يمنع super_admin من شاشات tenant
StudentRoute	student فقط — مثل AI Ideation
InvitationRoute	supervisor/student — ليس admin

5.3 إدارة الحالة (State Management)

لا يستخدم النظام Redux أو Zustand. تعتمد إدارة الحالة على:

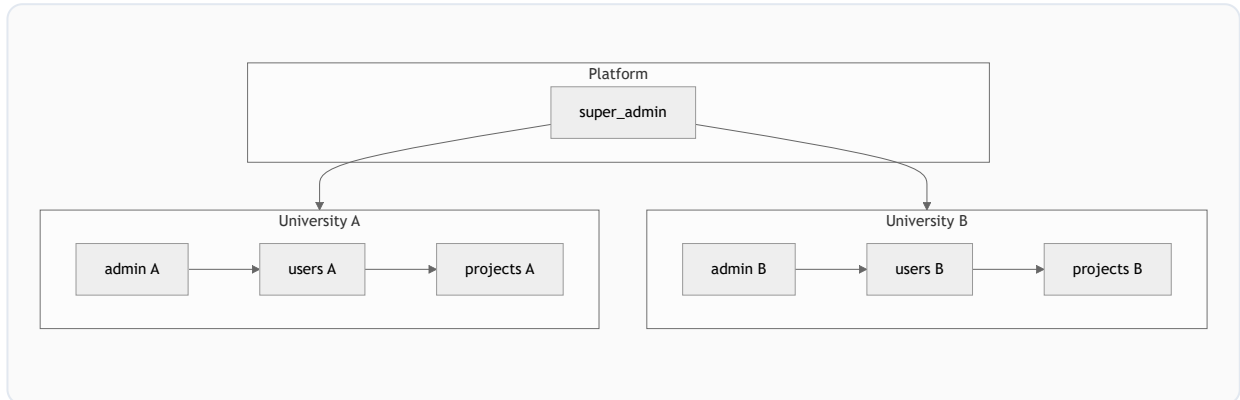
- **AuthContext:** المستخدم، sessionBlock، apiFetch، Token
- **LanguageContext:** العربية/الإنجليزية + RTL
- **ThemeContext:** Light/Dark + MUI Theme
- **Local State:** useState في كل Page للبيانات المؤقتة

هذا اختيار مناسب لحجم المشروع الحالي — يقلل التعقيد مع الحفاظ على وضوح تدفق البيانات.

6. معمارية Multi-Tenancy

6.1 النموذج: Shared Database, Shared Schema

جميع الجامعات تستخدم قاعدة بيانات واحدة وجداول مشتركة. يتم العزل عبر عمود `university_id` وليس عبر قواعد منفصلة.



6.2 TenantScope — آلية العزل

يُطبق `TenantScope` كـ `Global Scope` على نماذج `Project` و `Task` :

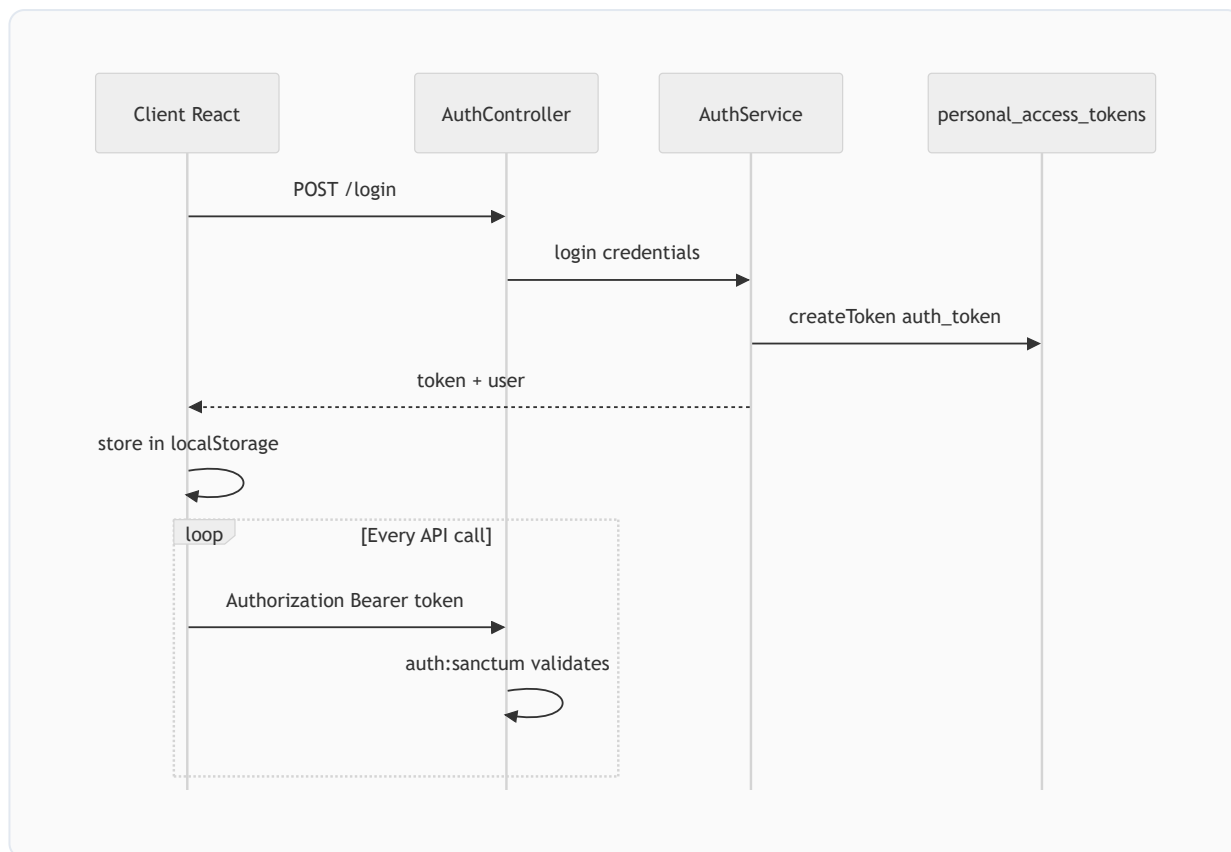
- كل استعلام يُفلتر تلقائياً بـ `university_id` للمستخدم الحالي.
- `super_admin` يتجاوز `Scope` ويرى كل الجامعات.
- محاولة الوصول لـ `ID` من جامعة أخرى تُرجع **404** (وليس 403) — أمان أفضل.

6.3 Admin الجامعة — لا جدول منفصل

مدير الجامعة = سجل في `users` حيث `roles.name = admin` و `users.university_id =` معرف الجامعة. لا يوجد جدول `admins`.

7. معمارية المصادقة والأمان

7.1 Sanctum — Token-Based Authentication



Role-Based Access Control (RBAC) 7.2

الدور	الآلية	النطاق
super_admin	role middleware + bypass TenantScope	Platform-wide
admin	role middleware + university_id filter	Tenant
supervisor	role + ProjectAccess	Tenant + supervised projects
student	role + ProjectAccess + AI routes	Tenant + owned/member projects

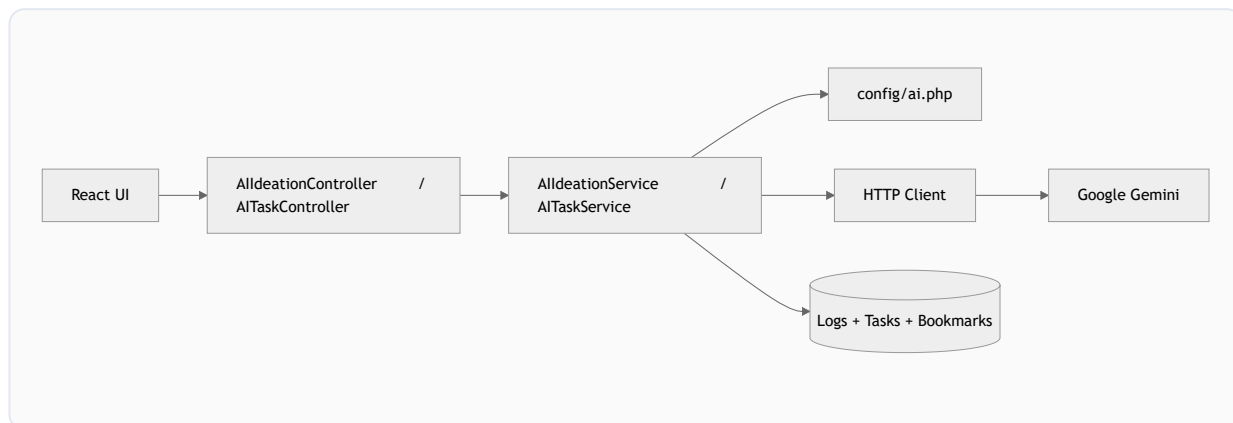
ProjectAccess 7.3 — صلاحيات المشروع

بالإضافة إلى RBAC، يُستخدم `ProjectAccess::canAccess()` للتحقق من أن المستخدم مالك المشروع، مشرفاً عليه، أو عضواً مقبولاً — قبل أي عملية على مشروع محدد.

8. معمارية الذكاء الاصطناعي

Integration Pattern 8.1

يُدمج AI ك **External Service** عبر **Service Layer — Controllers** لا تتصل بـ Gemini مباشرة.



8.2 ميزات AI

المخرجات	Service	Endpoint	الميزة
3 أفكار + bookmarks	AllIdeationService	POST /ai/suggest-projects	Project Ideation
10 مهام ai_generated	AITaskService	POST /projects/{id}/generate-tasks	Task Generation

8.3 حماية AI

- Rate limiting: ideation 10/hr, tasks 10/min
- Input sanitization قبل إرسال Gemini
- JSON mode + schema validation للاستجابة
- Logging في ideation_requests و task_generation_logs (بدون تخزين نص خام للاهتمامات)

9. معمارية البيانات

تعتمد طبقة البيانات على **Relational Model (MySQL)** مع 22 جدولاً نشطاً. العلاقات موثقة في

docs/PMS-ERD.dbm1

9.1 مجموعات الجداول

المجموعة	الجدول
Tenancy & Users	universities, roles, users, user_profiles, supervisor_universities
Projects	projects, project_members, tasks, comments, versions, activities, git_commits
Invitations	supervisor_invitations, student_invitations
AI	bookmarked_ideas, ideation_requests, task_generation_logs
Auth & System	personal_access_tokens, notifications, password_reset_requests

10. أنماط التصميم ومبادئ SOLID

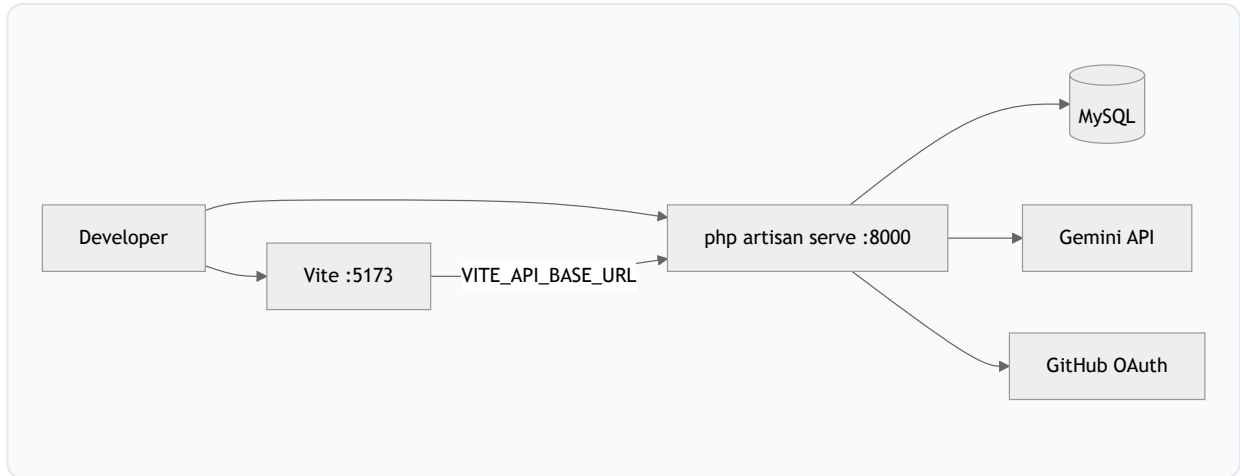
Design Patterns 10.1

التطبيق	Pattern
business logic فصل — */app/Services	Service Layer
persistence جزئي للـ ProjectRepository — abstraction	Repository
Laravel container — constructor injection	Dependency Injection
TenantScope — cross-cutting tenant filter	Global Scope
BelongsToUniversity, HasUniversityVisibility	Trait Composition
()ProjectAccess::canAccess	Facade-like Helper
AI Services: sanitize → prompt → HTTP → validate → persist	Template Method (structure)

SOLID 10.2

المبدأ	مثال في PMS
S — Single Responsibility	NotificationService للإشعارات فقط
O — Open/Closed	إضافة tenant لـ model جديد via trait دون تعديل TenantScope
L — Liskov Substitution	Services تمتد BaseService بسلوك response موحد
I — Interface Segregation	Services متخصصة بدل god service
D — Dependency Inversion	Controller → Service → Model (ليس HTTP في Controller)

11. النشر والبيئة التشغيلية



المكوّن	التقنية	الإعداد
Frontend	React 19 + Vite 7	npm run dev / build
Backend	+Laravel 10 + PHP 8.1	php artisan serve
Database	MySQL	php artisan migrate --seed
AI	Google Gemini	GEMINI_API_KEY in .env
Auth	Laravel Sanctum	personal_access_tokens table

12. المتطلبات غير الوظيفية

المتطلب	كيف يُحقق
الأمان	Sanctum, RBAC, TenantScope, input sanitization, rate limits
قابلية الصيانة	AI لـ Service Layer, migrations, feature tests
الأداء	Lazy loading frontend, Eloquent eager loading, API throttle
التوطين i18n	ar/en + RTL via LanguageContext
قابلية الاستخدام	MUI, role-based themes, responsive layout

13. مقارنة البدائل ومبررات الاختيار

القرار	البديل	لماذا اخترنا الحالي
Monolith API	Microservices	حجم المشروع لا يبرر تعقيد microservices
Service Layer	Fat Controllers	قابلية اختبار وصيانة أفضل
Shared DB Multi-Tenant	DB per tenant	أبسط في النشر والصيانة للجامعات
Sanctum Tokens	Session cookies	SPA + API stateless أفضل
Context API	Redux	كافي لحجم state الحالي
Gemini HTTP	Embedded ML	لا حاجة لبنية ML داخلية

14. القيود والتحسينات المستقبلية

قيود حالية:

- Repository Pattern غير مكتمل — معظم Services تستعلم Eloquent مباشرة
- لا Laravel Policies — الصلاحيات عبر ProjectAccess + Middleware
- Queue = sync — لا background jobs للـ AI
- Monolith — scaling أفقي محدود مقارنة بـ microservices

تحسينات مقترحة: Redis cache، Queue للـ Repository، CI/CD، AI requests، كامل، Real-time، notifications (WebSockets).

15. الخلاصة

يعتمد نظام PMS على معمارية **Client-Server** ثلاثية الطبقات مع **Service Layer** في Backend و**Component-Based SPA** في Frontend. يُحقق عزل Multi-Tenant عبر **Row-Level Isolation** و **TenantScope**. المصادقة **Stateless** عبر Sanctum. يُدمج الذكاء الاصطناعي كخدمة خارجية دون تغيير البنية الأساسية. هذه المعمارية توازن بين البساطة وقابلية الصيانة ومتطلبات **Multi-Tenant** لمشاريع التخرج الجامعية.

